AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 24: MCMC

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Monte Carlo methods, MCMC, Metropolis-Hastings, Gibbs sampling, Hamiltonian Monte Carlo, Convergence diagnostics, Variational Methods

2 Main Points

2.1 Monte Carlo

- Except in a few cases Bayesian inference is hard
  - It is easy when we have a conjugate prior giving us a simple posterior
  - But, when we want to model complicated situations our likelihood function usually doesn't have a conjugate prior
  - This means the posterior is not expressible as a simple probability distribution
  - Furthermore it is usually impossible to compute the marginal likelihood or evidence

$$\mathbb{P}[\mathcal{D}] = \sum_{\boldsymbol{\Theta}} \mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}]$$

  - The sum or integral over variables $\boldsymbol{\Theta}$ is often too large to make this feasible
  - You could try to obtain a histogram for the posterior, but if $\boldsymbol{\Theta}$ has many components the curse of dimensionality makes this difficult
- There is a simple answer which is to obtain samples from the posterior
  - We can use this to estimate expectations of any function, $g(\boldsymbol{\Theta})$, of $\boldsymbol{\Theta}$

$$\mathbb{E}_{\boldsymbol{\Theta}|\mathcal{D}}[g(\boldsymbol{\Theta})] \approx \frac{1}{n}\sum_{i=1}^{n} g(\boldsymbol{\Theta}_i)$$

    - $\boldsymbol{\Theta}_i$ are random deviates drawn from the posterior $\mathbb{P}[\boldsymbol{\Theta}_i|\mathcal{D}]$
    - (random deviates or random variates are values drawn from a probability distribution)
    - When the posterior distribution is well behaved (doesn't have a thick tail, so that $g(\boldsymbol{\Theta})$ has a finite variance) then the error in the approximation falls off as $1/\sqrt{n}$
    - For independent samples $\mathrm{Var}\big(\frac{1}{n}\sum_i g(\boldsymbol{\Theta}_i)\big) = \mathrm{Var}(g)/n$, whatever the dimension of $\boldsymbol{\Theta}$: the dimension only enters through $\mathrm{Var}(g)$
- The problem is you have to be able to sample random deviates from the posterior, which is proportional to $f(\mathcal{D}|\boldsymbol{\Theta})\, f(\boldsymbol{\Theta})$
- For simple distributions there are two classic solutions to generating random deviates

1. *Transformation Methods*
   - If we can compute the cumulative probability function
   $$F_X(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t$$
   and invert $F_X(x)$, then $X = F_X^{-1}(U)$ will be a random deviate with density f, where U is a uniform deviate between 0 and 1
   - This works because $\mathbb{P}[X \le x] = \mathbb{P}\left[F_X^{-1}(U) \le x\right] = \mathbb{P}[U \le F_X(x)] = F_X(x)$
   - E.g. for the exponential distribution $F_X(x) = 1 - \mathrm{e}^{-\lambda x}$, so $X = -\log(1-U)/\lambda$
   - This only works for a few classic distributions
2. *Rejection Method*
   - In the rejection methods we find another density $g_Y(y)$ that we can sample from that satisfies
   $$c\,g_Y(x) \ge f_X(x) \quad \text{for all } x$$
   - Then we generate a deviate $Y \sim g_Y$ and accept it with a probability $f_X(Y)/(c\,g_Y(Y))$ (draw U uniformly from $(0,1)$ and accept if $U\,c\,g_Y(Y) \le f_X(Y)$)
   - The probability of accepting is $\int g_Y(y)\,f_X(y)/(c\,g_Y(y))\,\mathrm{d}y = 1/c$, so on average we need c tries, or $c-1$ rejections, for each deviate
   - The accepted deviates have density f_X: the density of proposing y and accepting it is $g_Y(y)\,f_X(y)/(c\,g_Y(y)) = f_X(y)/c$, which is proportional to $f_X(y)$
   - This is a more general method, but is efficient only if we have a good approximating function $g_Y(y)$ (good in the sense that c is not too large)
   - Particularly for multivariate (i.e. high dimensional) random variables it is often very hard to find a good approximating function

2.2 Markov-Chain Monte Carlo (MCMC)

- Fortunately there is another method for generating random deviates which is very general
- This uses a Markov-Chain where we update an initial state of our system
- If we choose the dynamics correctly eventually we will reach a state that is correctly distributed
- Suppose we have a set of states, $\mathcal{S}$, and we want to choose a random sample from a distribution $\boldsymbol{\pi} = (\pi_i | i \in \mathcal{S})$
  - π_i is the probability of being in state i—we get to choose this (for Bayesian inference we will want to choose the posterior probability)
- Let M_{ij} be the transition probability from state j to state i
  - $\sum_i M_{ij} = 1$ (if we start in any state, j say, we have to end up in some state, possibly j itself)
  - $M_{ij} \ge 0$ (it is a probability)
  - Matrices of non-negative elements whose columns sum to one (with this convention) are known as **stochastic matrices**
- If we choose M_{ij} so that
  $$M_{ij}\,\pi_j = M_{ji}\,\pi_i$$
  - then after some time the probability of being in state i will be π_i (there are some mild conditions on M for this to be true)

- This condition is known as **detailed balance**
- Note that if we sum the detailed balance equation then

$$\sum_{j\in\mathcal{S}} M_{ij}\,\pi_j = \sum_{j\in\mathcal{S}} M_{ji}\,\pi_i = \pi_i$$

  - since $\sum_j M_{ji} = 1$
- Writing this in matrix form

$$\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$$

  - That is, $\boldsymbol{\pi}$ is an eigenvector of $\mathbf{M}$ with eigenvalue 1
- Because $\mathbf{M}$ is a stochastic matrix all eigenvalues have $|\lambda| \leq 1$
  - To prove this let $\boldsymbol{v}$ be an eigenvector of $\mathbf{M}$ with eigenvalue λ

$$\sum_{j\in\mathcal{S}} M_{ij}\,v_j = \lambda\,v_i$$

    taking the absolute value of both sides

$$\left|\sum_{j\in\mathcal{S}} M_{ij}\,v_j\right| = |\lambda\,v_i| = |\lambda|\,|v_i|$$

  - But

$$\left|\sum_{j\in\mathcal{S}} M_{ij}\,v_j\right| \leq \sum_{j\in\mathcal{S}} |M_{ij}\,v_j| = \sum_{j\in\mathcal{S}} M_{ij}\,|v_j|$$

    where we have used $M_{ij} \geq 0$ (and essentially $|a+b| \leq |a| + |b|$)
  - Thus

$$\sum_{j\in\mathcal{S}} M_{ij}\,|v_j| \geq |\lambda|\,|v_i|$$

  - Summing both sides with respect to i

$$\begin{aligned}
\sum_{i\in\mathcal{S}}\sum_{j\in\mathcal{S}} M_{ij}\,|v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i| \\
\sum_{j\in\mathcal{S}}\sum_{i\in\mathcal{S}} M_{ij}\,|v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i| \\
\sum_{j\in\mathcal{S}} |v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i|
\end{aligned}$$

    where we used $\sum_{i\in\mathcal{S}} M_{ij} = 1$
  - It follows that $|\lambda| \leq 1$
  - Let $\boldsymbol{p}(t)$ be a vector whose elements $p_i(t)$ equal the probability of being in state i at time t
  - If the set of eigenvectors spans the whole space, then any starting vector $\boldsymbol{p}(0)$ can be written in terms of the eigenvectors

$$\boldsymbol{p}(0) = \sum_i c_i\,\boldsymbol{v}_i$$

    (this is just a change of basis)
    - detailed balance guarantees this: with $\mathbf{D} = \mathrm{diag}(\boldsymbol{\pi})$ (all $\pi_i > 0$), the matrix $\mathbf{D}^{-1/2}\mathbf{M}\mathbf{D}^{1/2}$ has elements $M_{ij}\sqrt{\pi_j/\pi_i}$, which is symmetric by detailed balance, so $\mathbf{M}$ has real eigenvalues and a full set of eigenvectors

- The probability vector after one step is given by

$$\boldsymbol{p}(1) = \mathbf{M}\,\boldsymbol{p}(0) = \mathbf{M}\sum_i c_i\,\boldsymbol{v}_i = \sum_i c_i\,\mathbf{M}\,\boldsymbol{v}_i = \sum_i c_i\,\lambda_i\,\boldsymbol{v}_i$$

- After t steps

$$\boldsymbol{p}(t) = \mathbf{M}^t\,\boldsymbol{p}(0) = \sum_i c_i\,\lambda_i^t\,\boldsymbol{v}_i$$

- If $|\lambda_i| < 1$ then λ_i^t will shrink exponentially fast so that

$$\lim_{t\to\infty} \boldsymbol{p}(t) = \sum_{i:\lambda_i=1} c_i \boldsymbol{v}_i$$

- That is we converge onto the set of eigenvectors with eigenvalue 1
- But if detailed balance is conserved then $\boldsymbol{\pi}$ satisfies this
- There are some conditions to ensure that there is only one such eigenvector with eigenvalue 1 (and no eigenvalue -1)
  * We have to ensure that we can get from any state to any other through some series of transitions
  * We have to prevent periodic behaviour (e.g. if our set of states where from 1 to n and we could only move from state i to state $i-1$ or $i+1$ then if we started on an odd state we would always be on an odd state after an even number of moves)
  * It is very easy to ensure both conditions

- Now the problem is to choose the transition probabilities to satisfy detailed balance
- This is easy and can be done in different ways
- **Metropolis Algorithm**
  - A very easy way to achieve detailed balance is that starting from state j we propose a neighbouring state i
    - We have to make sure the probability, q, of proposing state i when in state j is the same as the probability of proposing state j when in state i (e.g. every state has the same number of neighbours, each chosen with equal probability)
  - We now move to state i if
    - $\pi_i \geq \pi_j$
    - or otherwise we move anyway with a probability π_i/π_j
  - Detailed balance: if $\pi_i \geq \pi_j$ then $M_{ij} = q$ and $M_{ji} = q\,\pi_j/\pi_i$, so $M_{ij}\,\pi_j = q\,\pi_j = M_{ji}\,\pi_i$ (and the case $\pi_i < \pi_j$ is the same with i and j swapped)
- **Metropolis-Hastings Algorithm**
  - Sometimes it is difficult to ensure that choosing state i from state j is the same as choosing state j from state i
    - As an example considering our states were non-negative integers, we move from state i to state $i-1$ and $i+1$ with probability $\frac{1}{2}$, but we have a problem at state 0 where we can't move to state $i-1$
  - We can modify the Metropolis algorithm. Suppose $p(i|j)$ is the probability of choosing state i starting in state j then let $r = p(j|i)\,\pi_i/(p(i|j)\,\pi_j)$
    - we accept the move if $r > 1$
    - or with a probability r

  - in practice we can just draw a random number U uniformly between 0 and 1 and accept the move if $U < r$
  - this is the same as the Metropolis algorithm if $p(j|i) = p(i|j)$

- We can equally well apply this to (multi-dimensional) continuous variables, $\boldsymbol{\theta}$
  - If $\boldsymbol{\theta}$ is our current set of parameters, we choose a new set of parameter $\boldsymbol{\theta}'$ with probability $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ and then accept this proposal if $r = (p(\boldsymbol{\theta}|\boldsymbol{\theta}')\,\pi(\boldsymbol{\theta}'))/(p(\boldsymbol{\theta}'|\boldsymbol{\theta})\,\pi(\boldsymbol{\theta})) \geq 1$ or otherwise with a probability r
  - We now choose $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ so that with high probability $\boldsymbol{\theta}'$ is close to $\boldsymbol{\theta}$, this ensures that there is a reasonably high acceptance rate
  - But the step size is a trade-off: tiny steps are almost always accepted but the chain explores very slowly, while huge steps are almost always rejected and the chain sits still

- For Bayes calculations then $\pi(\boldsymbol{\theta})$ would be our posterior $f(\boldsymbol{\theta}|\mathcal{D})$
  - MCMC has the nice property that the update depends only on the ratio

$$\frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta})} = \frac{f(\boldsymbol{\theta}'|\mathcal{D})}{f(\boldsymbol{\theta}|\mathcal{D})} = \frac{f(\mathcal{D}|\boldsymbol{\theta}')\,f(\boldsymbol{\theta}')}{f(\mathcal{D}|\boldsymbol{\theta})\,f(\boldsymbol{\theta})}$$

  - That is, the normalising factor $f(\mathcal{D})$ that appears in Bayes' rule cancels
  - This is important because this is usually incomputable (although we can use MCMC to estimate this)
  - There is also often another advantage of this ratio: if we only change one variable at a time then frequently we can compute the ratio much more efficiently than the full likelihood and priors
  - When this happens then it pays to choose a single variable, θ_i, at a time, choose a neighbour θ_i' from a distribution $p(\theta_i'|\theta_i)$ and decide whether to accept this update

- **Gibbs Sampling**
  - A special case of updating one variable at a time is to draw the new value θ_k' from its conditional distribution $\pi(\theta_k'|\boldsymbol{\theta}_{-k})$, where $\boldsymbol{\theta}_{-k}$ are all the other components, which are kept fixed
  - This is *Gibbs sampling*
  - It is Metropolis-Hastings with proposal $p(\boldsymbol{\theta}'|\boldsymbol{\theta}) = \pi(\theta_k'|\boldsymbol{\theta}_{-k})$. Writing $\pi(\boldsymbol{\theta}) = \pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})$,

$$r = \frac{\pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\theta_k'|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})}{\pi(\theta_k'|\boldsymbol{\theta}_{-k})\,\pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})} = 1$$

    so every move is accepted
  - It is useful when the conditional distributions are standard distributions that we can sample from (often because of conjugate priors), even though the joint posterior is not
  - When the parameters are strongly correlated, moving one at a time only allows small steps, and the chain mixes slowly

- **Convergence**
  - A problem with MCMC is that your initial parameter, $\boldsymbol{p}(0)$, is different from $\boldsymbol{\pi}$ so you have to wait some considerable time before your samples are from posterior distribution
  - As we saw earlier the convergence rate is determined by the second largest eigenvalue of the transition matrix $\mathbf{M}$ (we never explicit calculate $\mathbf{M}$ so we don't know what this eigenvalue is)

    - If it is very close to 1 then your convergence can be slow
  - Furthermore if your posterior distribution is broad you need lots of independent samples to accurately compute expectations
  - To obtain an independent sample we have to run our MCMC a long time
  - In practice you have to throw away some samples in a *burn-in* period
  - You can then average over all the remaining samples you have (including repetitions where you don't accept a move)
  - If you do this long enough then you should get samples that cover values of $\boldsymbol{\theta}$ with a high posterior probability
  - Successive samples are correlated, so n samples are worth fewer than n independent ones (the *effective sample size*)
  - MCMC is slow because of this
  - There are a lot of clever tricks to speed up convergence and decorrelation times
- Often MCMC seems daunting because of the apparent difficulties
  - But don't be put off
  - You only need to use the tricks for extremely complicated problems
  - Usually a simple implementation works fine
  - Modern computers are so fast that, for problems with a modest number of parameters, a simple random-walk Metropolis-Hastings sampler gives accurate answers in seconds
    - Convergence can be a problem in pathological cases—these occur when using MCMC to model some physical systems—but for many Bayesian problems the Markov Chain is *rapidly mixing* meaning the convergence time is not excessively long
    - Random walks do get slow when there are many parameters or the parameters are strongly correlated; that is what Hamiltonian Monte Carlo (below) is for

2.3 Checking Convergence

- We never know for certain that a chain has converged, but there are standard checks, and MCMC libraries carry them out for you
- Run several chains from widely different starting points and discard the burn-in of each
- Plot the traces (the value of each parameter against time): after burn-in the chains should overlap and look alike. A chain stuck in one region, or drifting, shows that something is wrong
- $\hat{R}$ (the Gelman–Rubin statistic) compares the variance between the chains with the variance within them. If the chains are all sampling the same distribution these agree and $\hat{R} \approx 1$; a common rule is to require $\hat{R} < 1.01$
- Successive samples are correlated. If the correlation between samples k steps apart is ρ_k, the mean of n samples has the variance of the mean of about

$$n_{\text{eff}} = \frac{n}{1 + 2\sum_{k=1}^{\infty} \rho_k}$$

  independent samples, the *effective sample size*. Report it with your estimates: the error in a posterior mean is roughly the posterior standard deviation divided by $\sqrt{n_{\text{eff}}}$

2.4 Hamiltonian Monte Carlo

- A random walk explores by diffusion: to cross a distance D with steps of size s takes about $(D/s)^2$ steps. When the posterior is a long narrow ridge (strongly correlated parameters) the step must be as small as the ridge is narrow, and progress along the ridge is very slow. The same happens in high dimensions
- Hamiltonian (or hybrid) Monte Carlo (HMC) uses the gradient of the log posterior to make long moves that are still accepted
- Treat $U(\boldsymbol{\theta}) = -\log \pi(\boldsymbol{\theta})$ as a landscape. As before this only needs likelihood times prior, since the evidence only adds a constant to U
- Introduce a fictitious momentum $\boldsymbol{p}$ of the same dimension and the energy

  $$H(\boldsymbol{\theta}, \boldsymbol{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\|\boldsymbol{p}\|^2$$

  The joint density $\propto \mathrm{e}^{-H} = \pi(\boldsymbol{\theta})\,\mathrm{e}^{-\|\boldsymbol{p}\|^2/2}$, so $\boldsymbol{\theta}$ and $\boldsymbol{p}$ are independent, $\boldsymbol{\theta}$ has the distribution we want and $\boldsymbol{p}$ is standard normal
- Each HMC step
  1. draws a fresh momentum $\boldsymbol{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  2. follows the frictionless motion of a ball in the landscape, $\dot{\boldsymbol{\theta}} = \boldsymbol{p}$, $\dot{\boldsymbol{p}} = -\boldsymbol{\nabla} U(\boldsymbol{\theta})$, for L small time steps of size ϵ using the *leapfrog* method (a half step in $\boldsymbol{p}$, alternate full steps in $\boldsymbol{\theta}$ and $\boldsymbol{p}$, and a final half step in $\boldsymbol{p}$)
  3. accepts the end point with probability $\min\bigl(1, \mathrm{e}^{H_{\text{start}} - H_{\text{end}}}\bigr)$
- The exact motion conserves H, so the acceptance probability would be 1; the leapfrog method makes small errors, and step 3 is a Metropolis–Hastings step that corrects for them, so the samples come exactly from π. (This relies on the leapfrog method being reversible and preserving volume, which it does)
- Because H is almost conserved, moves a long way across the distribution are almost always accepted
- In practice
  - HMC needs $\boldsymbol{\nabla} \log \pi(\boldsymbol{\theta})$; automatic differentiation (as used to train neural networks in PyTorch or JAX) provides it
  - it needs a step size ϵ and a number of steps L: too short a path and HMC is a random walk, too long and the path wastes time coming back. The *No-U-Turn Sampler* (NUTS; Hoffman and Gelman, 2014) chooses these automatically, stopping the path when it starts to turn back on itself
  - NUTS is the default sampler in probabilistic programming languages such as Stan, PyMC and NumPyro: you write down the generative story and get posterior samples
- In the lecture's example (a Gaussian with correlation 0.95, starting in the tail) 300 random-walk steps crawl along the ridge, while 15 HMC trajectories of 20 steps each, the same number of evaluations, cross it every time (Figure 1)

2.5 The Wider Landscape

- *Variational inference* fits an approximate posterior by optimisation (maximising the ELBO, see the notes on probabilistic inference and below). It is fast but only approximate

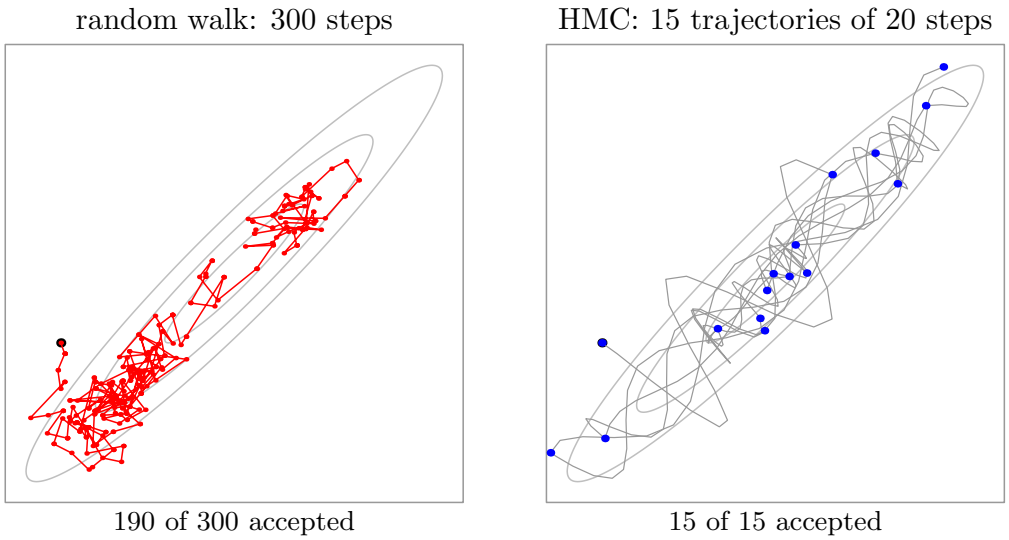


Figure 1: Random-walk Metropolis (left) and HMC (right) started at the same point on a Gaussian with correlation 0.95, with the same number of evaluations. Grey curves: HMC trajectories; blue: HMC samples

- *Stochastic-gradient MCMC* (e.g. stochastic-gradient Langevin dynamics) takes noisy gradient steps on minibatches of data, which makes MCMC possible for very large data sets and for Bayesian neural networks
- *Parallel tempering* and *sequential Monte Carlo* help when the posterior has several well-separated peaks, which a single chain rarely crosses between
- Diffusion models, which generate images in modern generative AI, draw their samples using Langevin-type dynamics, a close relative of MCMC

2.6 Variational Techniques

- This is a completely different approach to MCMC, but it tries to solve the same problem, but uses a different strategy
- This is again technically challenging and I don't expect you to memorise the derivation
- We saw in earlier lectures that we can cheat by seeking the parameters that maximise the posterior (the, so called, MAP solution)
- But this sacrifices all the probabilistic information
- In variational techniques we try to approximate the posterior with a simpler distribution
- That is, we approximate the posterior $f(\boldsymbol{\theta}|\mathcal{D})$ with a simpler distribution $q(\boldsymbol{\theta}|\boldsymbol{w})$ where we get choose $\boldsymbol{w}$ to make q close to f
- Usually we choose q to be a factorisable distribution

$$q(\boldsymbol{\theta}|\boldsymbol{w}) = \prod_i q(\theta_i|w_i)$$

- These techniques were first developed in the physics community and so come with a strange language

- In the *variational approximations* we consider the **variational free energy**

$$\Phi(\boldsymbol{w}) = \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log\left(\frac{q(\boldsymbol{\theta}|\boldsymbol{w})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

  - writing $f(\boldsymbol{\theta}, \mathcal{D}) = f(\boldsymbol{\theta}|\mathcal{D}) \, f(\mathcal{D})$ we can rearrange the variational free energy as

$$\begin{aligned}\Phi(\boldsymbol{w}) &= \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \left(\log\left(\frac{q(\boldsymbol{\theta}|\boldsymbol{w})}{f(\boldsymbol{\theta}|\mathcal{D})}\right) - \log(f(\mathcal{D}))\right) \mathrm{d}\boldsymbol{\theta} \\ &= \mathrm{KL}\big(q(\boldsymbol{\theta}|\boldsymbol{w}) \big\| f(\boldsymbol{\theta}|\mathcal{D})\big) - \log(f(\mathcal{D}))\end{aligned}$$

  - where $\mathrm{KL}\big(q(\boldsymbol{\theta}|\boldsymbol{w}) \big\| f(\boldsymbol{\theta}|\mathcal{D})\big)$ is the KL divergence that measures the "distance" between $q(\boldsymbol{\theta}|\boldsymbol{w})$ and $f(\boldsymbol{\theta}|\mathcal{D})$
  - the KL-divergence is non-negative and equal to zero only if the two distributions are the same (we showed this in the notes on probabilistic inference, and meet it again in the lecture on entropy)
  - The term $\log(f(\mathcal{D}))$ is the *marginal likelihood* or *evidence* and does not depend on the parameters $\boldsymbol{w}$
  - If we choose $\boldsymbol{w}$ to minimise $\Phi(\boldsymbol{w})$ then we minimise the KL-divergence and force $q(\boldsymbol{\theta}|\boldsymbol{w})$ to approximate the posterior $f(\boldsymbol{\theta}|\mathcal{D})$
  - The variational free energy can also be written as

$$\begin{aligned}\Phi(\boldsymbol{w}) &= -\int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(f(\boldsymbol{\theta}, \mathcal{D})) \, \mathrm{d}\boldsymbol{\theta} + \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(q(\boldsymbol{\theta}|\boldsymbol{w})) \, \mathrm{d}\boldsymbol{\theta} \\ &= -\big(L_q(\boldsymbol{w}) + H_q(\boldsymbol{w})\big)\end{aligned}$$

  - Minimising the variational free energy is equivalent to maximising $L_q(\boldsymbol{w}) + H_q(\boldsymbol{w})$
  - $L_q(\boldsymbol{w}) = \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(f(\boldsymbol{\theta}, \mathcal{D})) \, \mathrm{d}\boldsymbol{\theta}$ is the expected log of the joint probability of data and parameters (likelihood times prior), averaged over our approximate posterior $q(\boldsymbol{\theta}|\boldsymbol{w})$
  - $H_q(\boldsymbol{w}) = -\int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(q(\boldsymbol{\theta}|\boldsymbol{w})) \, \mathrm{d}\boldsymbol{\theta}$ is the entropy of our approximate posterior (this measures the uncertainty in $\boldsymbol{\theta}$)
  - We thus balance maximising the likelihood of the data with maximising the uncertainty in our approximate posterior
  - This means we fit the data, but guard against over fitting by allowing there to be a non-zero probability wherever it does not strongly contradict the data
  - In practice both $L_q(\boldsymbol{w})$ and $H_q(\boldsymbol{w})$ tend to be quite easy to compute
  - We are left with an optimisation problem for the parameters, $\boldsymbol{w}$, but this is usually quite quick to compute

3 Exercises

3.1 Transformation method

- Find a transformation of a uniform deviate U on $(0, 1)$ that gives a deviate with density $f(x) = 2x$ for $0 \leq x \leq 1$.

3.2 Rejection method

- We draw normal deviates $\mathcal{N}\big(x\big|0, 1\big)$ using Cauchy deviates with density $g(x) = 1/(\pi(1 + x^2))$ as proposals (as in the lecture). Find the smallest c such that $c\, g(x) \geq \mathcal{N}\big(x\big|0, 1\big)$ for all x, and the fraction of proposals that are accepted.

3.3 A two-state chain

- A chain has two states with $\boldsymbol{\pi} = (\frac{1}{4}, \frac{3}{4})$. Each state's only neighbour is the other state. Using the Metropolis algorithm, write down the transition matrix $\mathbf{M}$, check that $\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$, and find the other eigenvalue. How does $\boldsymbol{p}(t)$ approach $\boldsymbol{\pi}$?

3.4 Mysterious Disease

- Recall in the lecture on probabilistic inference, we defined $Z(t)$ to be the number of people that catch a disease on day t
- We assumed the growth rate is given by

$$\mathbb{P}[Z(t+1)] = \mathrm{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t) + Z(t-1) + Z(t-2)\right)\Big)$$

  - that is, people are infectious for three days

- The observed number of new people with the disease, $X(t)$, on day t is

$$\mathbb{P}[X(t) = k] = \mathrm{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k \left(1-p\right)^{Z(t)-k}$$

- Here is $X(t)$ for the first 30 days
  0,0,0,0,1,1,2,1,0,0,3,10,19,34,44,93,117,221,376,633,
  1021,1643,2701,4503,7414,12091,19999,33286,54612,90283

- Estimate p and r_0 from the data
- (If you prefer simulate your own data and estimate your parameters)

4 Experiments

4.1 Random walk against HMC

- Sample a Gaussian with correlation 0.95 by random-walk Metropolis and by HMC with the same number of evaluations, and compare their effective sample sizes

```
import numpy as np

rng = np.random.default_rng(0)
rho, d = 0.95, 2
P = np.linalg.inv(np.array([[1, rho], [rho, 1]]))   # precision matrix
U = lambda x: 0.5 * x @ P @ x                       # -log pi + const
gradU = lambda x: P @ x


def ess(x):
    """Effective sample size from the autocorrelations (summed to the first
    negative one)."""
    x = x - x.mean()
    acf = np.correlate(x, x, "full")[len(x) - 1:] / (x @ x)
    k = np.argmax(acf < 0)
    return len(x) / (1 + 2 * acf[1:k].sum())
```

```
budget = 30000                                  # evaluations for each
x, rw = np.zeros(d), []
for t in range(budget):                         # random-walk Metropolis
    y = x + 0.3 * rng.standard_normal(d)
    if np.log(rng.random()) < U(x) - U(y):
        x = y
    rw.append(x[0])

L, eps = 20, 0.15
x, hmc = np.zeros(d), []
for t in range(budget // L):                    # HMC, L gradients each
    p = rng.standard_normal(d)
    q, H0 = x.copy(), U(x) + 0.5 * p @ p
    p = p - 0.5 * eps * gradU(q)                # leapfrog
    for s in range(L):
        q = q + eps * p
        if s < L - 1:
            p = p - eps * gradU(q)
    p = p - 0.5 * eps * gradU(q)
    if np.log(rng.random()) < H0 - (U(q) + 0.5 * p @ p):
        x = q
    hmc.append(x[0])

for name, xs in [("random walk", np.array(rw)), ("HMC", np.array(hmc))]:
    print(name, len(xs), "samples, mean", round(xs.mean(), 3),
          "ESS", round(ess(xs)))
```

- (See the answers.)

4.2 Metropolis for the traffic rate

- The lecture's traffic example with a random-walk proposal $\mu' = \mu + s\,\epsilon$, $\epsilon \sim \mathcal{N}\big(\epsilon\big|0,1\big)$, compared with the exact posterior $\mathrm{Gamma}(\mu|\sum_i N_i, n)$

```
import numpy as np

rng = np.random.default_rng(0)
N = rng.poisson(5.0, size=20)           # cars on 20 days, true rate 5
n, S = len(N), N.sum()


def log_post(mu):                       # log(likelihood x prior), prior 1/mu
    return -n * mu + S * np.log(mu) - np.log(mu) if mu > 0 else -np.inf


for step in [0.05, 0.5, 5.0]:
    mu, samples, accepted = 1.0, [], 0
    for t in range(60000):
        mu_new = mu + step * rng.standard_normal()   # symmetric proposal
        if np.log(rng.random()) < log_post(mu_new) - log_post(mu):
            mu, accepted = mu_new, accepted + 1
        if t >= 10000:                  # discard burn-in
            samples.append(mu)          # count repeats when we don't move
    samples = np.array(samples)
```

```
    print(f"step {step:4}: accept {accepted/60000:.2f}  "
          f"mean {samples.mean():.3f}  sd {samples.std():.3f}")
print(f"exact Gamma({S},{n}): mean {S/n:.3f}  sd {np.sqrt(S)/n:.3f}")
```

- Note that we work with the log of the posterior (products of many probabilities underflow) and that a proposal $\mu' \le 0$ is simply rejected (its posterior is 0). Try changing the step size and the length of the run. (See the answers.)

5 Answers

5.1 Random walk against HMC

- Over three random seeds the 30000 random-walk samples have an effective sample size of 150–190, while the 1500 HMC samples have about 1500: essentially independent (the estimate is capped at n because successive HMC samples are slightly anti-correlated). For the same work HMC gives about ten times as many effective samples here, and the advantage grows with the dimension

5.2 Transformation method

- $F(x) = \int_0^x 2t\,\mathrm{d}t = x^2$, so $X = F^{-1}(U) = \sqrt{U}$

5.3 Rejection method

- We need $c \ge \mathcal{N}\big(x\big|0,1\big)\,/g(x) = \sqrt{\pi/2}\,(1+x^2)\,\mathrm{e}^{-x^2/2}$ for all x. Differentiating, $2x - x(1+x^2) = x(1-x^2) = 0$, so the maximum is at $x = \pm 1$, giving $c = \sqrt{2\pi}\,\mathrm{e}^{-1/2} \approx 1.52$ (the lecture uses 1.53)

- The acceptance probability is $1/c \approx 0.66$: about a third of the proposals are wasted

5.4 A two-state chain

- From state 1 we always propose state 2, and $\pi_2 > \pi_1$ so we always accept: $M_{21} = 1$, $M_{11} = 0$. From state 2 we propose state 1 and accept with probability $\pi_1/\pi_2 = \frac{1}{3}$: $M_{12} = \frac{1}{3}$, $M_{22} = \frac{2}{3}$

$$\mathsf{M} = \begin{pmatrix} 0 & \frac{1}{3} \\ 1 & \frac{2}{3} \end{pmatrix}, \qquad \mathsf{M}\boldsymbol{\pi} = \begin{pmatrix} \frac{1}{3}\cdot\frac{3}{4} \\ \frac{1}{4} + \frac{2}{3}\cdot\frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{3}{4} \end{pmatrix}$$

- The eigenvalues sum to the trace, $\frac{2}{3}$, so the other is $-\frac{1}{3}$. $\boldsymbol{p}(t) - \boldsymbol{\pi}$ shrinks by a factor of 3 each step and changes sign each step: the chain overshoots and oscillates as it converges

5.5 Metropolis for the traffic rate

- Running the code gives (the exact posterior has mean 5.300 and standard deviation 0.515)

step s	acceptance	mean	sd
0.05	0.97	5.278	0.496
0.5	0.71	5.300	0.510
5	0.13	5.296	0.506

- All three agree with the exact answer to within a few hundredths, but for different reasons: with $s = 0.05$ nearly every move is accepted but each moves only a little, so successive samples are strongly correlated; with $s = 5$ most proposals are rejected. A step comparable to the width of the posterior works best. With shorter runs the poor choices of s give visibly worse estimates

5.6 Mysterious Disease

- We need to decide on a prior
  - A reasonable prior for p is a Beta distribution perhaps with $a = b = 1$ (this is a uniform prior)
  - For r_0 we could use a Gamma prior with $a = 2$ and $b = 1$, which has a mean of $r_0 = 2$ (it seems to be considerably greater than 1)—we need to include $r_0 \, \mathrm{e}^{-r_0}$ as our prior (the normalisation is irrelevant)
- Now we have to choose proposal distributions for the new values of p and r_0
  - You could use $p' = p + U(-0.01, 0.01)$ and $r_0' = r_0 + U(-0.1, 0.1)$ where $U(a, b)$ is a uniform random deviate between a and b
  - These proposals are symmetric; a proposal outside the allowed range ($0 \le p \le 1$ and $r_0 \ge 0$) has zero posterior probability and is simply rejected
  - We could instead choose $p' \sim \mathrm{Beta}(10\,p, 10{-}10\,p)$ and $r_0' \sim r_0 \, \mathrm{Gamma}(5, 5) = \mathrm{Gamma}(5, 5/r_0)$, which never leave the allowed range and ensure that in expectation p' equals p and r_0' equals r_0 with small variations
    - Beware that there are two common definitions of Gamma distributions

$$\mathrm{Gamma}(z|a, b) = \frac{b^a}{\Gamma(a)} \, z^{a-1} \, \mathrm{e}^{-b\,z}$$

$$\mathrm{Gamma}^*(z|\alpha, \beta) = \frac{1}{\beta^\alpha \, \Gamma(\alpha)} z^{\alpha-1} \, \mathrm{e}^{-z/\beta}$$

    - You have to check which of these your libraries use (numpy's `rng.gamma(shape, scale)` uses the second)
    - These proposals are not symmetric, so we would need Metropolis-Hastings to ensure we get unbiased samples
- The variables $Z(t)$ are latent variables: we also have to sample them (e.g. by proposing changes to one $Z(t)$ at a time), and their samples estimate the actual number of cases of people with the virus
  - My data was generated with $p = 0.1$ and $r_0 = 2.5$
  - A check: with people infectious for three days the cases grow by a factor λ per day with $\lambda^3 = \frac{r_0}{3}(\lambda^2 + \lambda + 1)$. The data grow by a factor of about 1.65 a day, and $\lambda = 1.65$ gives $r_0 \approx 2.5$. The growth rate fixes r_0, but p is only determined by the early days, when the numbers are small